MeetAI

Detailed Project, Architecture & Implementation Guide

15-Day Vibe-Coding Portfolio Build

A practical implementation blueprint based on the uploaded MeetAI project specification, organized for development with Codex, Claude Code and Antigravity.

Target	Strong portfolio project
Build window	14–16 days (15-day recommended plan)
Core stack	React + TypeScript + FastAPI + PostgreSQL + WebSockets + Redis + Celery + OpenAI + Docker + Nginx
Primary outcome	Working, tested and deployed real-time AI meeting platform

Important: use AI to accelerate implementation, not to replace understanding. Every major subsystem should be explainable in an interview.

1. Project Overview

MeetAI is a full-stack real-time meeting platform designed around authentication, meeting management, live communication, asynchronous audio processing, AI-generated meeting intelligence, searchable history, testing and containerized deployment.

The uploaded specification defines the core technology choices as React/TypeScript, FastAPI, PostgreSQL, WebSockets, Redis, Celery, OpenAI and Docker/Nginx deployment.

The intended portfolio target is the core scope rather than the future-scope features. Do not expand into WebRTC video/audio, calendar integrations, advanced RBAC, speaker diarization, live AI assistance, vector search or analytics unless the core system is already complete.

Project success criteria

- Users can register, log in, refresh sessions, log out and manage profiles.
- Users can create, join, leave, cancel and archive meetings.
- Meeting participants can communicate in real time through WebSockets.
- Chat messages and meeting activity are persisted where appropriate.
- Audio can be uploaded and processed asynchronously through Celery/Redis.
- Transcripts, summaries, decisions and action items are stored and displayed.
- Meeting history and transcript content are searchable.
- The application is tested, containerized and deployed behind Nginx/HTTPS.

2. System Architecture

High-level architecture

```
Internet → Nginx → React frontend / FastAPI backend → PostgreSQL + Redis → Celery worker → OpenAI
```

The backend is intentionally separated into API, core/configuration, models, schemas, services, repositories, WebSocket handling and background workers. The frontend is separated into reusable components, pages, feature modules, hooks, services and types.

Component responsibilities

Component	Responsibility
React + TypeScript	User interface, routing, forms, meeting room, chat, transcript and summary views.
FastAPI	REST APIs, authentication, meeting operations, WebSocket endpoint and orchestration of background jobs.
PostgreSQL	Durable users, meetings, participants, messages, transcripts, summaries, action items and refresh-token data.
Redis	Celery broker and shared real-time infrastructure/pub-sub support where required.
Celery	Long-running audio/transcription/AI jobs outside the request-response path.
OpenAI	AI-powered transcription/meeting intelligence according to the selected implementation.
Docker Compose	Repeatable development and production service orchestration.
Nginx	Reverse proxy, static frontend delivery and HTTPS termination in deployment.

3. User Flow

Primary user journey

```
Register → Login → Dashboard → Create/Join Meeting → Meeting Room → Real-time Chat/Presence → Audio Upload → Processing → Transcript → AI Summary/Actions → Search/History
```

Detailed flow

- **Authentication:** User registers or logs in. The backend verifies credentials and returns the appropriate access/refresh session data.
- **Dashboard:** Authenticated user sees meeting history and can create or enter meetings.
- **Meeting creation:** Backend creates the meeting and owner/participant relationship.
- **Joining:** A second user joins through the meeting endpoint and then establishes a WebSocket connection.
- **Live meeting:** WebSocket events handle presence, chat, typing indicators and meeting state.
- **Audio processing:** Audio is uploaded through the API, validated, and queued instead of blocking the API worker.
- **AI pipeline:** Celery processes transcription and subsequent AI stages, storing results in PostgreSQL.
- **Review:** Users view transcript, summary, decisions and action items.
- **Search:** Meeting and transcript information can be found through the search/history interface.

4. User Communication & Real-Time Event Chart

Two-user meeting communication

```
User A Browser ■■WebSocket■■→ FastAPI ■■broadcast■■→ User B Browser ■ ■ ■■■■■ chat.message ■■■■■■ ■■■→ PostgreSQL (persist message) ■■■→ Redis/shared infrastructure when needed
```

Typical WebSocket events

Event	Direction	Purpose
connection.join	Client → Server	Authenticate/connect user to a meeting room.
presence.joined	Server → Room	Tell participants that a user joined.
presence.left	Server → Room	Tell participants that a user left.
chat.message	Client → Server → Room	Send and broadcast a chat message; persist it.
typing.start	Client → Room	Show typing indicator.
typing.stop	Client → Room	Remove typing indicator.
meeting.state	Server → Room	Synchronize relevant meeting state.
error	Server → Client	Return a controlled WebSocket error.

5. AI / Audio Processing Flow

```
Audio Upload → FastAPI Validation → Processing Record → Redis/Celery Queue → Celery Worker → Transcription → Transcript Storage → AI Analysis → Summary/Decisions/Action Items → PostgreSQL → React
```

Why asynchronous processing?

Audio processing and AI calls can be long-running. The specification therefore separates these jobs from the API request path so normal API workers remain responsive.

Processing state model

State	Meaning
UPLOADED	File accepted and processing record created.
PROCESSING	Background worker is actively processing the meeting audio.

State	Meaning
COMPLETED	Transcript/AI outputs were successfully persisted.
FAILED	Processing stopped after an error; error information should be available for diagnosis/retry.

6. Database Design

Table	Purpose / key relationship
users	Application users and authentication identity.
meetings	Meeting metadata and ownership.
meeting_participants	Many-to-many relationship between users and meetings.
messages	Persisted meeting chat messages.
transcripts	Transcript metadata/result associated with a meeting.
transcript_segments	Segment-level transcript content.
meeting_summaries	Structured meeting summary/decision information.
action_items	Follow-up tasks associated with a meeting/summary.
refresh_tokens	Controlled refresh-session/token data.

Relationship view

```
User 1 ■■< Meeting 1 ■■< MeetingParticipant >■■ 1 User Meeting 1 ■■< Messages Meeting 1 ■■ 1 Transcript
■■< TranscriptSegments Meeting 1 ■■< Summary ■■< ActionItems
```

7. Technology Stack

Layer	Technology	Why / role
Frontend	React + TypeScript + Vite	Component-based UI with type safety and fast development.
Styling	Tailwind CSS	Rapid consistent responsive UI development.
Routing	React Router	Protected/authenticated page flows and navigation.
Backend	FastAPI	Typed REST APIs, validation and WebSocket support.
ORM	SQLAlchemy 2.x	Database models and relationships.
Validation	Pydantic	Request/response schemas and structured data validation.
Database	PostgreSQL	Durable relational application data.
Migrations	Alembic	Version-controlled schema changes.
Real-time	WebSockets	Live meeting chat, presence and events.
Cache / broker	Redis	Celery broker and shared real-time infrastructure.
Workers	Celery	Asynchronous audio/AI processing.
AI	OpenAI	Transcription/meeting intelligence stage.
Containers	Docker / Docker Compose	Repeatable local and production environments.
Proxy	Nginx	Reverse proxy, static delivery and HTTPS.
Testing	Pytest + frontend/e2e tooling	Backend, WebSocket, worker, UI and end-to-end validation.

8. Recommended Repository Structure

```
meetai/ ■■■ backend/ ■ ■■■ app/ ■ ■ ■■■ api/ ■ ■ ■■■ core/ ■ ■ ■■■ models/ ■ ■ ■■■ schemas/ ■ ■
■■■ services/ ■ ■ ■■■ repositories/ ■ ■ ■■■ websocket/ ■ ■ ■■■ workers/ ■ ■■■ tests/ ■■■ frontend/ ■
■■■ src/ ■ ■ ■■■ components/ ■ ■ ■■■ pages/ ■ ■ ■■■ features/ ■ ■ ■■■ hooks/ ■ ■ ■■■ services/ ■ ■
■■■ types/ ■ ■■■ tests/ ■■■ nginx/ ■■■ docker-compose.yml ■■■ .env.example ■■■ README.md ■■■ docs/
```

9. 15-Day Implementation Plan

Day	Deliverable	Definition of done
1	Foundation	Repo, React, FastAPI, PostgreSQL, Redis, Docker, env config, health check.
2	Database	SQLAlchemy models, relationships, Alembic migrations, schemas and repository/service base.
3	Authentication	Register, login, refresh, logout, current user, protected routes.
4	Meetings	Create/list/detail/join/leave/cancel/archive plus dashboard UI.
5	WebSocket foundation	Authenticated meeting socket, rooms, lifecycle, broadcast and cleanup.
6	Chat + presence	Real-time chat, typing, join/leave presence, message persistence.
7	Redis + reliability	Shared real-time support as needed, reconnect handling and failure cleanup.
8	Audio + Celery	Validated upload, processing state and asynchronous job creation.
9	Transcription	Worker stage produces/persists transcript and segments.
10	AI intelligence	Structured summary, decisions and action items persisted.
11	History + search	Meeting history, transcript viewing and searchable content.
12	Security hardening	Auth audit, validation, rate limits, file controls, retry/idempotency and error handling.
13	Testing	Backend, DB, WebSocket, worker, frontend and critical E2E flow tests.
14	Deployment	Production Docker, Nginx, HTTPS, cloud services and health checks.
15	Portfolio polish	UI polish, README, architecture/database diagrams, screenshots, metrics and interview prep.

10. Vibe-Coding Workflow

The most reliable workflow is not 'build everything'. Work feature-by-feature and force the AI to explain architecture before large changes.

Recommended loop

```
1. YOU define the feature 2. AI proposes plan/files 3. AI implements one feature 4. YOU run it locally 5.
AI diagnoses failures 6. AI adds tests 7. Claude Code reviews architecture/security 8. Antigravity reviews
frontend/UX where applicable 9. YOU understand the implementation 10. Git commit
```

Codex usage

- Best for feature implementation, tests, debugging, refactoring and terminal/Git workflows.
- Ask for small, scoped changes and require tests for bug fixes.
- After each subsystem, ask it to explain the important files and data flow.

Claude Code usage

- Best for repository-wide architecture review and multi-file refactors.
- Use it for security/reliability audits before making changes.
- Ask it to identify root causes before asking it to rewrite code.

Antigravity usage

- Best for frontend implementation, visual consistency, responsive behavior and UX review.

• Use it after functionality works so visual work does not destabilize core backend development.

11. Reusable AI Prompts

Feature implementation prompt

```
Implement [FEATURE] in the existing MeetAI architecture. First inspect the relevant files and explain the implementation plan. Then make the smallest coherent set of changes. Follow the existing service/repository/schema patterns, add validation and error handling, and add tests. Do not rewrite unrelated files. After implementation, list changed files and explain the data flow.
```

Architecture review prompt

```
Review the current MeetAI repository as a senior engineer. Check architecture, security, data consistency, error handling, scalability and maintainability. Do not change code yet. Return Critical/High/Medium issues with file references, why each matters and a recommended fix.
```

Debugging prompt

```
Diagnose this failure in the existing MeetAI implementation. Do not guess and do not rewrite the subsystem. Trace the request/event/job flow, identify the root cause, explain it, propose the minimal fix, then implement it and add a regression test.
```

Interview prompt

```
Based only on the current MeetAI codebase, generate 30 technical interview questions with concise model answers. Cover FastAPI, PostgreSQL, JWT, WebSockets, Redis, Celery, AI processing, Docker, security and deployment. Do not ask about features that are not actually implemented.
```

12. Security & Reliability Checklist

• Hash passwords; never store plaintext passwords.

• Use short-lived access tokens and controlled refresh-token handling.

• Authorize access to meetings/resources at the API layer.

• Validate request payloads and file uploads; enforce file-size limits.

• Configure CORS deliberately rather than allowing everything in production.

• Add rate limiting where required for sensitive endpoints and real-time interactions.

• Keep secrets in environment/configuration, not source control.

• Do not expose raw stack traces or sensitive provider errors to clients.

• Use timeouts and retries for external AI calls.

• Make background processing idempotent so duplicate jobs do not create duplicate results.

• Handle WebSocket disconnects and reconnects cleanly.

• Use transactions appropriately around durable state changes.

• Add pagination to history/search endpoints.

13. Testing Strategy

Test layer	What to test
Unit	Services, validation, utility logic, authentication helpers.
API	Auth, meetings, search, audio upload, authorization and error cases.
Database	Relationships, constraints, persistence and transactional behavior.
WebSocket	Join/leave, chat, presence, typing, auth failure, disconnect/reconnect.
Background jobs	Task creation, retries, failure states, duplicate-job/idempotency behavior.

Test layer	What to test
Frontend	Login, dashboard, meeting room, chat, search and key loading/error states.
E2E	Register → login → create meeting → second user joins → chat → audio → transcript → AI → search.

14. Deployment Plan

Production service set

- Frontend container or static build served through Nginx.
- FastAPI application container.
- PostgreSQL database.
- Redis service.
- Celery worker service.
- Nginx reverse proxy with HTTPS.
- Environment variables for database, Redis, JWT and AI provider configuration.

Deployment verification

- Open the deployed frontend and complete authentication.
- Create a meeting and join from a second browser/session.
- Verify real-time chat and presence.
- Upload an allowed audio file and observe processing state changes.
- Verify transcript/summary/action-item persistence.
- Run search.
- Check logs and health endpoints.
- Confirm secrets are not committed to Git.

15. Portfolio Metrics to Measure

Do not invent performance numbers for your resume. Measure them from your deployed/tested system.

Metric	How to use it
API p95 latency	Measure representative endpoints such as login, meeting detail and search.
WebSocket latency	Measure message round-trip or event propagation in a controlled test.
Concurrent participants	Test a defined number of simultaneous meeting connections and record the observed result.
AI processing time	Measure time from upload/job creation to completed transcript/AI result.
Test coverage	Report only the actual coverage produced by your test suite.
Deployment availability	If you measure it over time, report the actual observed period/result.

16. Interview Preparation

You should be able to explain these flows without looking at the code:

- JWT access/refresh authentication flow.
- Meeting creation and participant authorization.
- WebSocket connection lifecycle and room broadcasting.

- How Redis fits into real-time/background infrastructure.
- Why Celery is used for audio/AI processing.
- How duplicate background jobs are prevented or made safe.
- How AI failures/retries are handled.
- How transcript and summary data are modeled.
- How search is implemented and indexed.
- How Docker services communicate.
- Why Nginx is used in deployment.
- How the system would scale if traffic increased.

A strong interview explanation

```
“MeetAI is a React/FastAPI real-time meeting platform. REST APIs handle authentication and meeting
management, WebSockets handle live communication, PostgreSQL stores durable application data, Redis
supports background and real-time infrastructure, and Celery moves long-running audio/AI processing out of
the API request path. The application is containerized with Docker and deployed behind Nginx.”
```

17. Scope Control

Build the specified core first. The uploaded document identifies several items as future scope. They should not consume the 15-day build unless the core is already stable.

- Native WebRTC video/audio.
- Google Calendar or Microsoft Outlook integrations.
- Advanced organization RBAC.
- Speaker diarization.
- Live AI meeting assistant.
- Vector/embedding search.
- Analytics dashboard.
- Enterprise SSO.

18. Final Definition of Done

Area	Done when...
Authentication	Register/login/refresh/logout/profile work with authorization.
Meetings	Create/join/leave/cancel/archive and history work.
Real-time	Two or more users can chat and see presence with reconnect handling.
AI	Audio reaches background processing and produces stored transcript + structured AI output.
Search	Meeting/transcript content can be searched with authorization and pagination.
Security	Input/file validation, token controls, rate limits and safe errors are addressed.
Testing	Critical backend, WebSocket, worker, frontend and E2E flows are covered.
Deployment	Dockerized services run in production behind Nginx/HTTPS.
Portfolio	README, architecture diagram, database diagram, screenshots and real metrics are ready.
Understanding	You can explain every major subsystem and its trade-offs.

Recommended pace: 4–6 focused hours/day. Prioritize a working core over adding future-scope features. The best portfolio version is not the one with the most code; it is the one that works, is tested, deployed, and can be explained.